

0.75.1. Arrivé en (0, 0), le A devient B — de base, ailleurs

Le 0.75 avec un autre départ : le A part de (10, 8), et devient toujours B en (0, 0). Il faut aller à gauche, puis monter.

1. Ce qu'il faut comprendre

C'est le 0.75, avec un seul changement : le **départ**. Le A part de **(10, 8)**, au milieu de l'écran, au lieu de (10, 0).

La case de transformation ne change pas : **(0, 0)**, le coin en haut à gauche. La condition est la même : `if (x == 0 && y == 0)`.

Le chemin est plus long : la flèche **gauche** amène le A en (0, 8), au bord gauche. Là, `x == 0` est vrai, mais `y == 0` est faux (y vaut 8) : il reste un A. Puis la flèche **haut** le fait monter : 7, 6... et en (0, 0), les deux sont vrais. Il devient B.

C'est ce que montre `&&` : arriver dans la bonne colonne ne suffit pas, il faut aussi la bonne ligne. Regarde les nombres en bas : `000 008` au bord gauche, encore un A ; `000 000` au coin, un B.

C'est la version de **base** : une seule lettre, un autre départ. Le 0.75.2 met les deux ensemble.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1823 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 10;    // le changement : le départ, (10, 8), au milieu
```

2. UNE VARIABLE — Y

```
uint8_t y = 8;
```

3. UNE VARIABLE — LETTRE

```
uint8_t lettre = 0;    // 0 = A, 1 = B
```

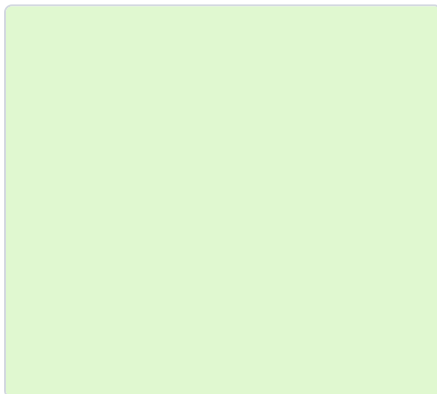
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();
        deplace_croix(x, y, ALPHABET[lettre], 250);
        nombre(0, 17, x);                // la position, en direct
        nombre(4, 17, y);

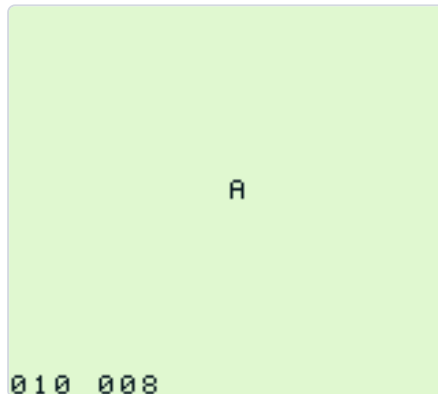
        // La même case qu'au 0.75 : (0, 0), le coin en haut à gauche.
        //
        //   en (0, 8) : x == 0 vrai, y == 0 faux → encore un A (il faut monter)
        //   en (0, 0) : les deux vrais          → le A devient B
        // Sur une ligne : if (x == 0 && y == 0) lettre = 1;
        // Avec des accolades, la même chose :
        if (x == 0 && y == 0) {
            lettre = 1;
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Un A au milieu de l'écran ; la flèche gauche l'amène au bord (0, 8), encore un A ; la flèche haut le fait monter jusqu'au coin (0, 0), où il devient B.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ au bord (0, 8), c'est encore un A

✓ en (0, 0), il est devenu B

✓ la position s'affiche : 000 000 (« 000 000 »)

Ensuite : **0.75.2. Arrivé en (0, 0), le A devient B — doublé, deux positions** — niveau 0 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++